Algorithms for Bioinformatics
Bioinformatics
2025/2026

Biopython: freely available Python tools for computational molecular biology and bioinformatics

Miriam Santos
[dCC] @ Faculty of Sciences University of Porto

Outline

- **Bioinformatics Toolbox: Biopython**
  - What is Biopython
  - Biopython Features
  - Installing Biopython
  - Processing Sequences with Biopython
  - Sequence Alignments in Biopython
  - Using BLAST through Biopython

What is Biopython?

Its the largest and most popular bioinformatics package for Python!

- A collection of Python modules that provide functions to deal with DNA, RNA, and proteins sequence operations.

- It provides a lot of parsers to read all major genetic databases like GenBank, SwissProt, etc., as well as wrappers/interfaces to run other popular bioinformatics software/tools such as NCBI BLASTN, Entrez, etc., inside the Python environment.

- Provides **standardized access** to bioinformatics resources and enables **high-quality, reusable modules and scripts.**

https://biopython.org

Biopython Features

- Interpreted, interactive, and **object-oriented!**
- Supports FASTA, PDB, GenBank, BLAST, and several other formats.
- Can connect to **BioSQL** databases (SQL tables to store sequences and annotations)
- Code to dal with popular **online services and databases**, including NCBI services (BLAST, Entrez, PubMed) ema ExPASY services (SwissProt, Prosite)
- Interfaces to common bioinformatics programs (**local services**), such as standalone BLAST from NCBI, ClustalW alignment program, EMBOSS command line tools.

Biopython Features

- A standard sequence class that deals with sequences, ids on sequences, and sequence features.
- Tools for performing common operations on sequences, such as translation, transcription and weight calculations.
- Code for dealing with alignments, including a standard way to create and deal with substitution matrices.
- Code to perform classification using KNN, Naïve Bayes, or SVMs.
- Extensive documentation and help with using the modules.

Installing Biopython

- The latest version of Biopython **(1.87)** is required to work with Python >=3.10 (https://pypi.org/project/biopython/).

- Currently, version 3.13 is recommende: (https://github.com/biopython/biopython/blob/master/README.rst).

  - The developers recommend the installation through PyPI. You can first create a conda environment and then use **pip** to install biopython:

```
conda create -n biopython-tutorial python=3.13 pip
conda activate biopython-tutorial
pip install biopython
```

Installing Biopython

```
(biopython-tutorial) miriamsantos@Miriams-MacBook-Pro ~ % python –version

Python 3.13.2
```

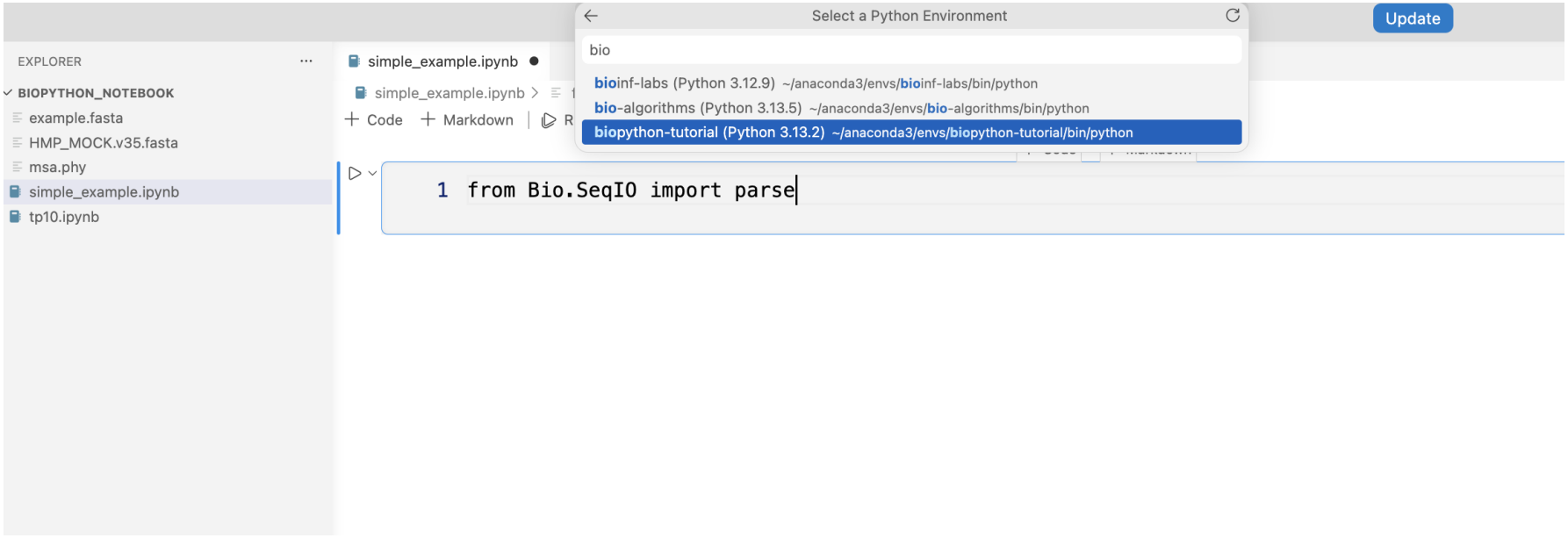
```
conda list
```

```
# Name                    Version                   Build  Channel
appnope                   0.1.4              pyhd8ed1ab_1    conda-forge
asttokens                 3.0.0              pyhd8ed1ab_1    conda-forge
biopython                 1.85                     pypi_0    pypi
bzip2                     1.0.8                h6c40b1e_6
```

Getting Started: A Simple Example


```
>sp|P25730|FMS1_ECOLI CS1 fimbrial subunit A precursor (CS1 pilin)
MKLKKTIGAMALATLFATMGASAVEKTISVTASVDPTVDLLQSDGSALPNSVALTYSPAV
NNFEAHTINTVVHTNDSDKGVVVKLSADPVLSNVLNPTLQIPVSVNFAGKPLSTTGITID
SNDLNFASSGVNKVSSTQKLSIHADATRVTGGALTAGQYQGLVSIILTKSTTTTTTKGT

>sp|P15488|FMS3_ECOLI CS3 fimbrial subunit A precursor (CS3 pilin)
MLKIKYLLIGLSLSAMSSYSLAAAGPTLTKELALNVLSPAALDATWAPQDNLTLSNTGVS
NTLVGVLTLSNTSIDTVSIASTNVSDTSKNGTVTFAHETNNSASFATTISTDNANITLDK
NAGNTIVKTTNGSQLPTNLPLKFITTEGNEHLVSGNYRANITITSTIKGGGTKKGTTDKK
```



Getting Started: A Simple Example

+ Code + Markdown

```python
from Bio.SeqIO import parse
from Bio.SeqRecord import SeqRecord
from Bio.Seq import Seq
```

✓ 0.0s Python

```python
file = open("example.fasta")
```

✓ 0.0s Python

```python
records = parse(file, "fasta")
```

✓ 0.0s Python

```python
for record in records:
    print(f"ID: {record.id}")
    print(f"Description: {record.description}")
    print(f"Sequence Data: {record.seq}")
```

✓ 0.0s Python

```
ID: sp|P25730|FMS1_ECOLI
Description: sp|P25730|FMS1_ECOLI CS1 fimbrial subunit A precursor (CS1 pilin)
Sequence Data: MKLKKTIGAMALATLFATMGASAVEKTISVTASVDPTVDLLQSDGSALPNSVALTYSPAVNNFEAHTINTVVHTNDSDKGVVVKLSADPVLSNVLNPTLQIPVSVN
ID: sp|P15488|FMS3_ECOLI
Description: sp|P15488|FMS3_ECOLI CS3 fimbrial subunit A precursor (CS3 pilin)
Sequence Data: MLKIKYLLIGLSLSAMSSYSLAAAGPTLTKELALNVLSPAALDATWAPQDNLTLSNTGVSNTLVGVLTLSNTSIDTVSIASTNVSDTSKNGTVTFAHETNNSASFA
```

Processing Sequences with Biopython

- Sequences in Biopython are represented by the **Seq** class, defined in the **Bio.Seq** module

```
from Bio.Seq import Seq
seq = Seq("AGCT")
Print(seq)
```

- The most important difference between **Seq objects and standard Python strings** is that they **have different methods**.
- Yet, **Seq** objects support many of the same methods as a plain string!

```
seq_string = Seq("AGCTAGCT")
print(seq_string[0])
print(seq_string[0:2])
print(len(seq_string))
seq_string.count("A")
seq_string[::-1]
```

```
seq1 = Seq("ACGT")
seq2 = Seq("AACCGG")
seq1 + seq2
```

- **Bio.SeqUtils** also offers miscellaneous functions for dealing with sequences: gc_fraction, molecular_weight, six_frame_translation:

```
from Bio.SeqUtils import gc_fraction, molecular_weight, six_frame_translations

my_seq = Seq("GATCGATGGGCCTATATAGGATCGAAAATCGC")
```

```
print(gc_fraction(my_seq))
print(molecular_weight(my_seq))
print(six_frame_translations(my_seq))
```

```
0.46875
9977.3766
```

```
GC_Frame: a:10 t:7 g:9 c:6
Sequence: gatcgatggg ... cgaaaatcgc, 32 nt, 46.88 %GC


1/1
  S  M  G  L  Y  R  I  E  N  R
 I  D  G  P  I  *  D  R  K  S
D  R  W  A  Y  I  G  S  K  I
gatcgatgggcctatataggatcgaaaatcgc   46 %
ctagctacccggatatatcctagcttttagcg
D  I  P  R  Y  L  I  S  F  R
 R  H  A  *  I  P  D  F  I  A
  I  S  P  G  I  Y  S  R  F  D
```

https://github.com/biopython/biopython/blob/master/Bio/SeqUtils/__init__.py
https://biopython.org/docs/latest/api/Bio.SeqUtils.html

- Biopython does not check the sequence contents and **will not raise an exception** if for example you concatenate a protein sequence and a DNA sequence (**which is likely a mistake**):

```
protein_seq = Seq("EVRNAK")
dna_seq = Seq("ACGT")
protein_seq + dna_seq
```

```
Seq('EVRNAKACGT')
```

Complement and Reverse Complement

- For nucleotide sequences, you can obtain the complement or reverse complement using built-in methods:

```
my_seq.complement()
my_seq.reverse_complement()
```

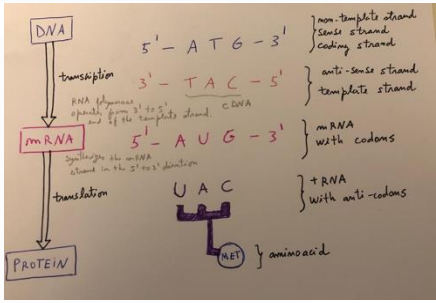
- The reverse/complement of a protein is biologically meaningless… but allowed.

```
protein_seq = Seq("EVRNAK")
protein_seq.complement()

>> Seq('EBYNTM')
```

  - "E" is not a valid IUPAC ambiguity code for nucleotides, so it was not complemented. However, "V" means "A", "C" or "G" and has complement "B", and so on.

Transcription



DNA coding strand (aka Crick strand, strand $+1$)

```
5'  ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG  3'
    ||||||||||||||||||||||||||||||||||||||||
3'  TACCGGTAACATTACCCGGCGACTTTCCCACGGGCTATC  5'
```

DNA template strand (aka Watson strand, strand -1)

Transcription of this DNA sequence produces the following RNA sequence:

```
5'  AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG  3'
```

Single-stranded messenger RNA

The actual biological transcription process works from the template strand, doing a reverse complement (TCAG $\rightarrow$ CUGA) to give the mRNA. However, in Biopython and bioinformatics in general, we typically work directly with the coding strand because this means we can get the mRNA sequence just by switching T $\rightarrow$ U.

```
coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
coding_dna.transcribe()

>> Seq('AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG')
```

Back Translation

- The **Seq** also includes a back-transcription method for going from the mRNA to the coding strand of the DNA.

```
mrna = Seq("AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG")

print(mrna)
print(mrna.back_transcribe())
```

```
AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG

ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG
```

Translation and Translation Tables

- We can translate DNA or RNA into the corresponding protein sequence:

```
coding_dna.translate()
mrna.translate()

>> Seq('MAIVMGR*KGAR*')
```

- The translation tables in Biopython are based on those from the NCBI. By default, the standard genetic code is used (NCBI id table 1), but we can change it:

```
coding_dna.translate(table="Vertebrate Mitochondrial")

>> Seq('MAIVMGRWKGAR*')
```

```
coding_dna.translate(table=2)

>> Seq('MAIVMGRWKGAR*')
```

Translation and Translation Tables

- We can also specify whether we wish to stop reading when we encounter the first stop codon (as happens in nature):

```
coding_dna.translate()

>> Seq('MAIVMGR*KGAR*')
```

```
coding_dna.translate(to_stop=True)

>> Seq('MAIVMGR')
```

Sequence I/O Operations

- Bio.SeqIO provides a module to read and write sequences from and to a file. The parsed sequence is presented to the user through the **SeqRecord** object.
  - Bio.SeqRecord holds meta-information of the sequence as well as the sequence data itself:
    - **seq:** It is the sequence itself, typically a **Seq** object;
    - **id:** The primary ID to identify the sequence (string) (like an accession number)
    - **name:** A "common" name for the sequence (string);
    - **description:** A human readable description for the sequence (string)
    - (…)
- When working with simple FASTA files, you can skip this. If you're working with GenBank or EMBL files, this part is quite important!

https://biopython.org/docs/latest/Tutorial/chapter_seq_annot.html

Parsing or Reading Sequences

Bio.SeqIO.parse() is used to read in sequence data as SeqRecord objects and expects two arguments:

1. A handle to read the data from or a filename.
2. The expected sequence format (specified as a lowercase string)

Bio.SeqIO.parse() returns an iterator which gives SeqRecord objects. If you're dealing with files which contain a single record, it is best to used Bio.SeqIO.read() which takes the same arguments. It returns a SeqRecord.

```
from Bio import SeqIO

for seq_record in SeqIO.parse("ls_orchid.fasta", "fasta"):
        print(seq_record.id)
        print(repr(seq_record.seq))
        print(len(seq_record))
```

Parsing or Reading Sequences

- If instead you wanted to load a GenBank format file:

```
from Bio import SeqIO

for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank"):
        print(seq_record.id)
        print(repr(seq_record.seq))
        print(len(seq_record))
```

- If you wanted to read a file in another file format, you would need to change the format string as appropriate: for example “swiss” for SwissProt or “embl” for EMBL text files.

https://biopython.org/wiki/SeqIO
https://biopython.org/docs/latest/api/Bio.SeqIO.html#

Sequence Alignments in Biopython

- In Biopython, all sequence alignments are represented by an **Alignment object.**
- Alignment objects can be obtained by **parsing the output of alignment software** such as CLUSTAL or BLAST.
- Or by **using Biopython's pairwise sequence aligner**, which aligns two sequences to each other.

https://biopython.org/docs/latest/Tutorial/chapter_align.html

Pairwise Sequence Alignment in Biopython

- The **Bio.Align** module contains the **PairwiseAligner** class for global and local alignments using the Needleman-Wunsch and Smith-Waterman algorithms, among others.
- To create **pairwise** alignments, we create a **PairwiseAligner** object to store the parameters used for the alignment. We can use **aligner.score** to get the alignment score or the **aligner.align** that returns a **PairwiseAlignments** object that tell us how many alignments were found and their scores:

```
from Bio import Align

aligner = Align.PairwiseAligner()

target = "GAACTTT"
query = "GATTT"
score = aligner.score(target, query)
alignments = aligner.align(target, query)

print(score)
>> 5.0

for alignment in alignments:
        print(alignment)
```

```
target            0 GAACTTT 7
                  0 ||--||| 7
query             0 GA--TTT 5

target            0 GAACTTT 7
                  0 |-|-||| 7
query             0 G-A-TTT 5
```

Pairwise Sequence Alignment in Biopython

- **By default, a global pairwise alignment is performed**, which finds the optimal alignment over the whole length of the target and query.

- Instead, a local alignment will find the subsequence of target and query with the highest alignment score. Local alignments can be generated by setting **aligner.mode to "local".**

```
aligner.mode = "local"
target = "AGAACTC"
query = "GAACT"
score = aligner.score(target, query)


print(score)
>> 5.0

for alignment in alignments:
        print(alignment)
```

```
target          1 GAACT 6
                0 ||||| 5
query           0 GAACT 5
```

Pairwise Sequence Alignment in Biopython

- The **PairwiseAligner** object stores all alignment parameters, which can be viewed by printing the object:

```
print(aligner)
```

```
Pairwise sequence aligner with parameters
  wildcard: None
  match_score: 1.000000
  mismatch_score: 0.000000
  target_internal_open_gap_score: 0.000000
  target_internal_extend_gap_score: 0.000000
  target_left_open_gap_score: 0.000000
  target_left_extend_gap_score: 0.000000
  target_right_open_gap_score: 0.000000
  target_right_extend_gap_score: 0.000000
  query_internal_open_gap_score: 0.000000
  query_internal_extend_gap_score: 0.000000
  query_left_open_gap_score: 0.000000
  query_left_extend_gap_score: 0.000000
  query_right_open_gap_score: 0.000000
  query_right_extend_gap_score: 0.000000
  mode: local
```

Pairwise Sequence Alignment in Biopython

- Depending on the mode, a **PairwiseAligner** object automatically chooses the appropriate algorithm to use for pairwise sequence alignment. **This attribute is read-only.**

```
aligner.mode=“global”
aligner.algorithm

>> 'Needleman-Wunsch'
```

```
aligner.mode=“local”
aligner.algorithm

>> ‘Smith-Waterman'
```

Substitution Scores

- Substitution scores can be defined in two ways:
  - By specifying match and mismatch scores, setting the **match** and **mismatch** attributes of the **PairwiseAligner**.

```
aligner = Align.PairwiseAligner()


aligner.match_score = 1.0
aligner.mismatch_score = -2.0
aligner.gap_score = -2.5


score = aligner.score("ACGT", "ACAT")
print(score)
```

Substitution Scores

- Substitution scores can be defined in two ways:

  - By using the **substitution_matrix** attribute. It is also possible to load some substitutions matrices available from Biopython.

```
from Bio.Align import substitution_matrices

matrix = substitution_matrices.load("BLOSUM62")
aligner.substitution_matrix = matrix

score = aligner.score("ACDQ", "ACDQ")
print(score)
```

Substitution Scores

- Substitution scores can be defined in two ways:

  - By using the **substitution_matrix** attribute. It is also possible to load some substitutions matrices available from Biopython.

```
substitution_matrices.load()
```

```
['BENNER22', 'BENNER6', 'BENNER74', 'BLASTN', 'BLASTP',
'BLOSUM45', 'BLOSUM50', 'BLOSUM62', 'BLOSUM80', 'BLOSUM90',
'DAYHOFF', 'FENG', 'GENETIC', 'GONNET1992', 'HOXD70',
'JOHNSON', 'JONES', 'LEVIN', 'MCLACHLAN', 'MDM78',
'MEGABLAST', 'NUC.4.4', 'PAM250', 'PAM30', 'PAM70', 'RAO',
'RISLER', 'SCHNEIDER', 'STR', 'TRANS']
```

Substitution Scores

```
1 matrix = substitution_matrices.load("BLOSUM62")
2 print(matrix)
```

✓ 0.0s

```
#  Matrix made by matblas from blosum62.iij
#  * column uses minimum score
#  BLOSUM Clustered Scoring Matrix in 1/2 Bit Units
#  Blocks Database = /data/blocks_5.0/blocks.dat
#  Cluster Percentage: >= 62
#  Entropy =   0.6979, Expected =  -0.5209
   A   R   N   D   C   Q   E   G   H   I   L   K   M   F   P   S   T   W   Y   V   B   Z   X   *
A  4.0 -1.0 -2.0 -2.0  0.0 -1.0 -1.0  0.0 -2.0 -1.0 -1.0 -1.0 -1.0 -2.0 -1.0  1.0  0.0 -3.0 -2.0  0.0 -2.0 -1.0  0.0 -4.0
R -1.0  5.0  0.0 -2.0 -3.0  1.0  0.0 -2.0  0.0 -3.0 -2.0  2.0 -1.0 -3.0 -2.0 -1.0 -1.0 -3.0 -2.0 -3.0 -1.0  0.0 -1.0 -4.0
N -2.0  0.0  6.0  1.0 -3.0  0.0  0.0  0.0  1.0 -3.0 -3.0  0.0 -2.0 -3.0 -2.0  1.0  0.0 -4.0 -2.0 -3.0  3.0  0.0 -1.0 -4.0
D -2.0 -2.0  1.0  6.0 -3.0  0.0  2.0 -1.0 -1.0 -3.0 -4.0 -1.0 -3.0 -3.0 -1.0  0.0 -1.0 -4.0 -3.0 -3.0  4.0  1.0 -1.0 -4.0
C  0.0 -3.0 -3.0 -3.0  9.0 -3.0 -4.0 -3.0 -3.0 -1.0 -1.0 -3.0 -1.0 -2.0 -3.0 -1.0 -1.0 -2.0 -2.0 -1.0 -3.0 -3.0 -2.0 -4.0
Q -1.0  1.0  0.0  0.0 -3.0  5.0  2.0 -2.0  0.0 -3.0 -2.0  1.0  0.0 -3.0 -1.0  0.0 -1.0 -2.0 -1.0 -2.0  0.0  3.0 -1.0 -4.0
E -1.0  0.0  0.0  2.0 -4.0  2.0  5.0 -2.0  0.0 -3.0 -3.0  1.0 -2.0 -3.0 -1.0  0.0 -1.0 -3.0 -2.0 -2.0  1.0  4.0 -1.0 -4.0
G  0.0 -2.0  0.0 -1.0 -3.0 -2.0 -2.0  6.0 -2.0 -4.0 -4.0 -2.0 -3.0 -3.0 -2.0  0.0 -2.0 -2.0 -3.0 -3.0 -1.0 -2.0 -1.0 -4.0
H -2.0  0.0  1.0 -1.0 -3.0  0.0  0.0 -2.0  8.0 -3.0 -3.0 -1.0 -2.0 -1.0 -2.0 -1.0 -2.0 -2.0  2.0 -3.0  0.0  0.0 -1.0 -4.0
I -1.0 -3.0 -3.0 -3.0 -1.0 -3.0 -3.0 -4.0 -3.0  4.0  2.0 -3.0  1.0  0.0 -3.0 -2.0 -1.0 -3.0 -1.0  3.0 -3.0 -3.0 -1.0 -4.0
L -1.0 -2.0 -3.0 -4.0 -1.0 -2.0 -3.0 -4.0 -3.0  2.0  4.0 -2.0  2.0  0.0 -3.0 -2.0 -1.0 -2.0 -1.0  1.0 -4.0 -3.0 -1.0 -4.0
K -1.0  2.0  0.0 -1.0 -3.0  1.0  1.0 -2.0 -1.0 -3.0 -2.0  5.0 -1.0 -3.0 -1.0  0.0 -1.0 -3.0 -2.0 -2.0  0.0  1.0 -1.0 -4.0
M -1.0 -1.0 -2.0 -3.0 -1.0  0.0 -2.0 -3.0 -2.0  1.0  2.0 -1.0  5.0  0.0 -2.0 -1.0 -1.0 -1.0 -1.0  1.0 -3.0 -1.0 -1.0 -4.0
F -2.0 -3.0 -3.0 -3.0 -2.0 -3.0 -3.0 -3.0 -1.0  0.0  0.0 -3.0  0.0  6.0 -4.0 -2.0 -2.0  1.0  3.0 -1.0 -3.0 -3.0 -1.0 -4.0
P -1.0 -2.0 -2.0 -1.0 -3.0 -1.0 -1.0 -2.0 -2.0 -3.0 -3.0 -1.0 -2.0 -4.0  7.0 -1.0 -1.0 -4.0 -3.0 -2.0 -2.0 -1.0 -2.0 -4.0
S  1.0 -1.0  1.0  0.0 -1.0  0.0  0.0  0.0 -1.0 -2.0 -2.0  0.0 -1.0 -2.0 -1.0  4.0  1.0 -3.0 -2.0 -2.0  0.0  0.0  0.0 -4.0
T  0.0 -1.0  0.0 -1.0 -1.0 -1.0 -1.0 -2.0 -2.0 -1.0 -1.0 -1.0 -1.0 -2.0 -1.0  1.0  5.0 -2.0 -2.0  0.0 -1.0 -1.0  0.0 -4.0
W -3.0 -3.0 -4.0 -4.0 -2.0 -2.0 -3.0 -2.0 -2.0 -3.0 -2.0 -3.0 -1.0  1.0 -4.0 -3.0 -2.0 11.0  2.0 -3.0 -4.0 -3.0 -2.0 -4.0
Y -2.0 -2.0 -2.0 -3.0 -2.0 -1.0 -2.0 -3.0  2.0 -1.0 -1.0 -2.0 -1.0  3.0 -3.0 -2.0 -2.0  2.0  7.0 -1.0 -3.0 -2.0 -1.0 -4.0
V  0.0 -3.0 -3.0 -3.0 -1.0 -2.0 -2.0 -3.0 -3.0  3.0  1.0 -2.0  1.0 -1.0 -2.0 -2.0  0.0 -3.0 -1.0  4.0 -3.0 -2.0 -1.0 -4.0
B -2.0 -1.0  3.0  4.0 -3.0  0.0  1.0 -1.0  0.0 -3.0 -4.0  0.0 -3.0 -3.0 -2.0  0.0 -1.0 -4.0 -3.0 -3.0  4.0  1.0 -1.0 -4.0
Z -1.0  0.0  0.0  1.0 -3.0  3.0  4.0 -2.0  0.0 -3.0 -3.0  1.0 -1.0 -3.0 -1.0  0.0 -1.0 -3.0 -2.0 -2.0  1.0  4.0 -1.0 -4.0
X  0.0 -1.0 -1.0 -1.0 -2.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -2.0  0.0  0.0 -2.0 -1.0 -1.0 -1.0 -1.0 -1.0 -4.0
```

- You can either run BLAST **locally** (on your own machine) or **remotely** (online, on another machine, typically the NCBI servers).

- To call the **online version** of BLAST, we call the function **qblast** from the **Bio.Blast module.**

The NCBI guidelines state:

1. Do not contact the server more often than once every 10 seconds.
2. Do not poll for any single RID more often than once a minute.
3. Use the URL parameter email and tool, so that the NCBI can contact you if there is a problem.
4. Run scripts weekends or between 9 pm and 5 am Eastern time on weekdays if more than 50 searches will be submitted.

- The **qblast** function has 3 non-optional arguments:

```
def qblast(
    program,
    database,
    sequence,
```

blastn, blastp, blastx, tblast, tblastx

databases to search against
https://blast.ncbi.nlm.nih.gov/doc/blast-help/blastdatabases.html#blastdatabases

string of your query sequence

```
fasta_string = open("m_cold.fasta").read()
result_stream = Blast.qblast("blastn", "nt", fasta_string
```

```
record = SeqIO.read("m_cold.fasta", "fasta")
result_stream = Blast.qblast("blastn", "nt", record.seq
```

The default settings on the NCBI BLAST website are not the same as the defaults of QBLAST. If you get different results, check the parameters: https://github.com/biopython/biopython/blob/master/Bio/Blast/NCBIWWW.py

Saving and Parsing BLAST results

- The **qblast** returns the BLAST results as a XML format. You can save a local copy of the output file:

```
from Bio import Blast
from Bio import SeqIO

record = SeqIO.read("m_cold.fasta", "fasta")
result_stream = Blast.qblast("blastn", "nt", record.seq)

with open("my_blast.xml", "wb") as out_stream:
        out_stream.write(result_stream.read())
result_stream.close()
```

- After this, the results are in the **my_blast.xml**. Now we can open it again to parse it more clearly!

```
result_stream = open("my_blast.xml", "rb")
```

Saving and Parsing BLAST results

- Now that we have a data stream, we can easily parse the output. **If you expect a single BLAST result (i.e., you used a single query):**

```
result_stream = open("my_blast.xml", "rb")
blast_record = Blast.read(result_stream)
```

- **If you have lots of results (i.e., multiple query sequences):**

```
result_stream = open("my_blast.xml", "rb")
blast_records = Blast.parse(result_stream)
```

Saving and Parsing BLAST results


```
with Blast.parse("my_blast.xml") as blast_records:
        print(blast_records)
```

```
Program: BLASTN 2.2.27+
     db: refseq_rna

  Query: 42291 (length=61)
         mystery_seq
   Hits: ----  -----  ----------------------------------------------------------
            #  # HSP  ID + description
         ----  -----  ----------------------------------------------------------
            0      1  gi|262205317|ref|NR_030195.1|  Homo sapiens microRNA 52...
            1      1  gi|301171311|ref|NR_035856.1|  Pan troglodytes microRNA...
            2      1  gi|270133242|ref|NR_032573.1|  Macaca mulatta microRNA ...
            3      2  gi|301171322|ref|NR_035857.1|  Pan troglodytes microRNA...
            4      1  gi|301171267|ref|NR_035851.1|  Pan troglodytes microRNA...
            5      2  gi|262205330|ref|NR_030198.1|  Homo sapiens microRNA 52...
            6      1  gi|262205302|ref|NR_030191.1|  Homo sapiens microRNA 51...
            7      1  gi|301171259|ref|NR_035850.1|  Pan troglodytes microRNA...
            8      1  gi|262205451|ref|NR_030222.1|  Homo sapiens microRNA 51...
            9      2  gi|301171447|ref|NR_035871.1|  Pan troglodytes microRNA...
           10      1  gi|301171276|ref|NR_035852.1|  Pan troglodytes microRNA...
           11      1  gi|262205290|ref|NR_030188.1|  Homo sapiens microRNA 51...
           12      1  gi|301171354|ref|NR_035860.1|  Pan troglodytes microRNA...
           13      1  gi|262205281|ref|NR_030186.1|  Homo sapiens microRNA 52...
           14      2  gi|262205298|ref|NR_030190.1|  Homo sapiens microRNA 52...
           15      1  gi|301171394|ref|NR_035865.1|  Pan troglodytes microRNA...
           16      1  gi|262205429|ref|NR_030218.1|  Homo sapiens microRNA 51...
           17      1  gi|262205423|ref|NR_030217.1|  Homo sapiens microRNA 52...
           18      1  gi|301171401|ref|NR_035866.1|  Pan troglodytes microRNA...
           19      1  gi|270133247|ref|NR_032574.1|  Macaca mulatta microRNA ...
           20      1  gi|262205309|ref|NR_030193.1|  Homo sapiens microRNA 52...
           21      2  gi|270132717|ref|NR_032716.1|  Macaca mulatta microRNA ...
           22      2  gi|301171437|ref|NR_035870.1|  Pan troglodytes microRNA...
           23      2  gi|270133306|ref|NR_032587.1|  Macaca mulatta microRNA ...
           24      2  gi|301171428|ref|NR_035869.1|  Pan troglodytes microRNA...
           25      1  gi|301171211|ref|NR_035845.1|  Pan troglodytes microRNA...
           26      2  gi|301171153|ref|NR_035838.1|  Pan troglodytes microRNA...
           27      2  gi|301171146|ref|NR_035837.1|  Pan troglodytes microRNA...
           28      2  gi|270133254|ref|NR_032575.1|  Macaca mulatta microRNA ...
           29      2  gi|262205445|ref|NR_030221.1|  Homo sapiens microRNA 51...
           ~~~
           97      1  gi|356517317|ref|XM_003527287.1|  PREDICTED: Glycine ma...
           98      1  gi|297814701|ref|XM_002875188.1|  Arabidopsis lyrata su...
           99      1  gi|397513516|ref|XM_003827011.1|  PREDICTED: Pan panisc...
```

- Instead of opening the file yourself, you can provide the file name.
- Biopython opens the file for you and closes it as soon as the file is not needed anymore (saving resources).

Bibliography

- Official git repository for Biopython: https://github.com/biopython/biopython/tree/master
- Biopython: Python Tools for Computational Molecular Biology: https://biopython.org
- Biopython Tutorial and Cookbook: https://biopython.org/docs/latest/Tutorial/index.html
- Notebooks: https://github.com/tiagoantao/biopython-notebook

Algorithms for Bioinformatics
Bioinformatics
2025/2026

Biopython: freely available Python tools for computational molecular biology and bioinformatics

Miriam Santos
[dCC] @ Faculty of Sciences University of Porto